

GeoLog — Plataforma de Telemetria Logística e Persistência Poliglota

Relatório Técnico — Desafio Integrador

Aluno: Josenilton Cirne Ramalho Neto

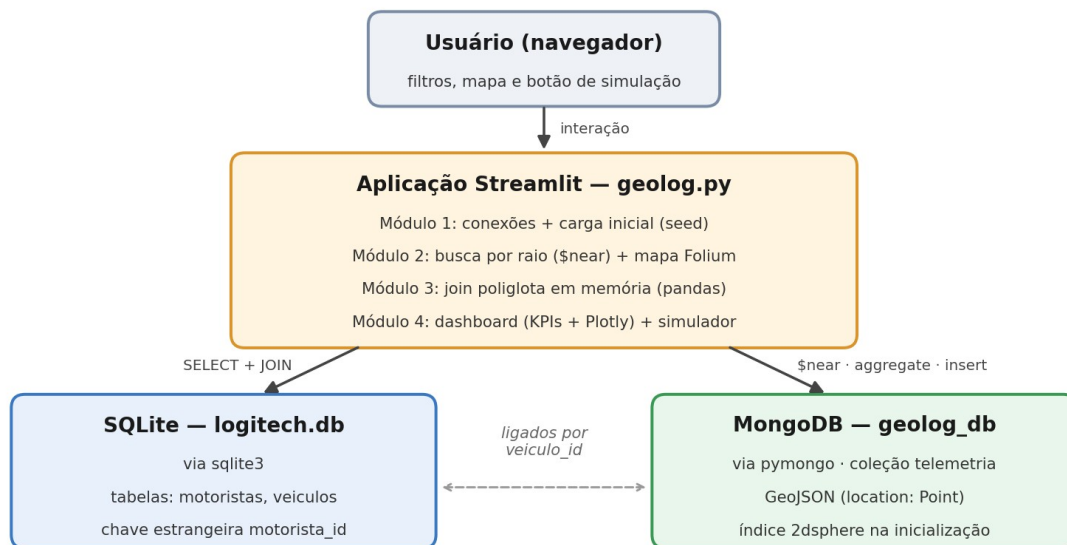
Código no GitHub: github.com/joseniltoncirne/Implementacao-e-Geren-de-banco-de-dados/tree/main/desafio-geolog

Observação: este desafio foi feito individualmente. Sou aluno novo na turma e não estive presente na aula em que os grupos foram formados, por isso ainda não tive contato com os colegas para montar um grupo.

1. Objetivo

Construir, num único arquivo Streamlit (geolog.py), uma plataforma para a transportadora LogiTech Express acompanhar sua frota em João Pessoa, guardando cada tipo de dado no banco mais adequado: os cadastros no SQLite e a telemetria dos caminhões no MongoDB.

2. Arquitetura poliglota



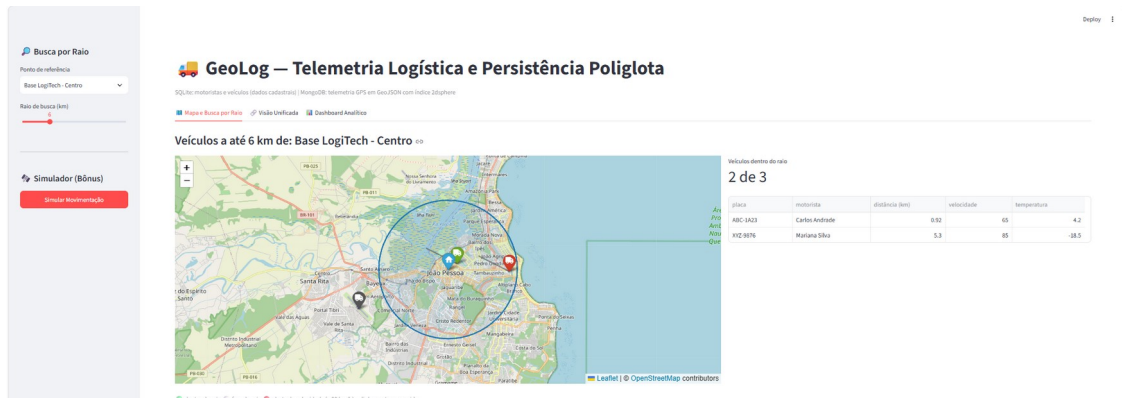
- **SQLite (logitech.db):** motoristas e veículos. São poucos dados, sempre no mesmo formato e ligados entre si (cada veículo tem um motorista, com chave estrangeira), por isso ficam num banco relacional.
- **MongoDB (geolog_db.telemetria):** as leituras de GPS, temperatura e velocidade. Chegam o tempo todo, em grande quantidade, e precisam de busca por localização; por isso ficam em documentos GeoJSON com índice 2dsphere.
- **Ligação entre os dois:** o campo `veiculo_id`. A aplicação busca cada parte no seu banco e junta tudo em memória com pandas.

3. Como cada módulo foi feito

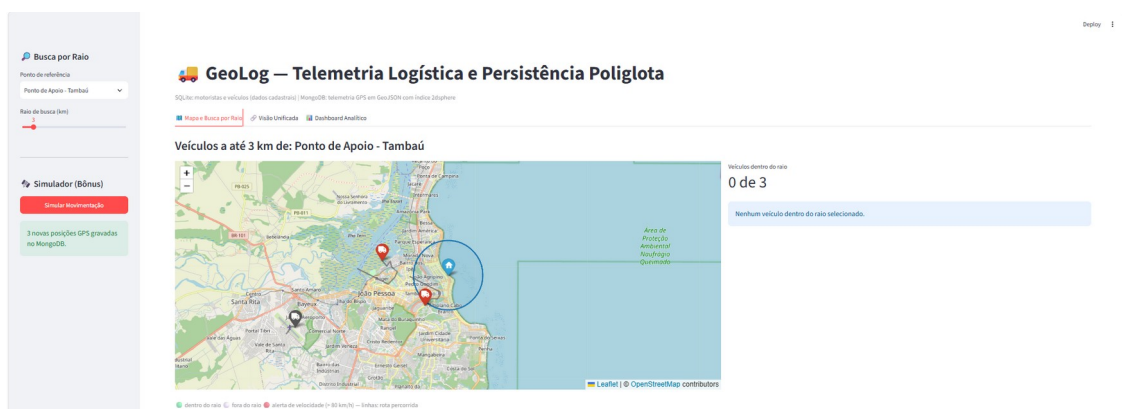
- **Módulo 1 – Persistência e carga inicial:** ao abrir, a aplicação cria as tabelas no SQLite (sqlite3) e a coleção no MongoDB (pymongo), cria o índice 2dsphere no campo location e carrega os dados sugeridos no enunciado. Para os gráficos terem histórico, cada caminhão ganha algumas leituras anteriores com pequenas variações.
- **Módulo 2 – Busca por raio:** o usuário escolhe um ponto de apoio (ou digita uma coordenada) e o raio em km. Um pipeline de agregação pega a última leitura de cada caminhão e o operador \$near com \$maxDistance filtra os que estão dentro do raio. O mapa Folium mostra o círculo do raio, a rota de cada caminhão e marcadores coloridos (verde: dentro do raio; cinza: fora; vermelho: acima de 80 km/h).
- **Módulo 3 – Visão unificada:** um SELECT com JOIN no SQLite traz motorista e placa; o MongoDB traz a última leitura de cada veículo; o pandas junta as duas partes pelo `veiculo_id`.
- **Módulo 4 – Dashboard:** indicadores de frotas ativas, temperatura média da carga e alertas de velocidade (> 80 km/h), gráfico de temperatura ao longo do tempo por veículo, gráfico do status dos motoristas e uma tabela de métricas calculada por um pipeline de agregação no MongoDB.

- **Bônus – Simular Movimentação:** o botão grava no MongoDB uma nova posição para cada caminhão, com pequena variação aleatória, e a tela se atualiza sozinha (mapa, tabelas e gráficos), sem reiniciar a aplicação.

4. Telas da aplicação



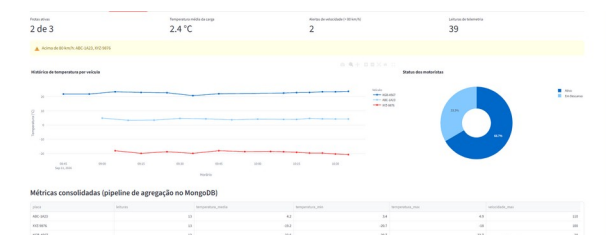
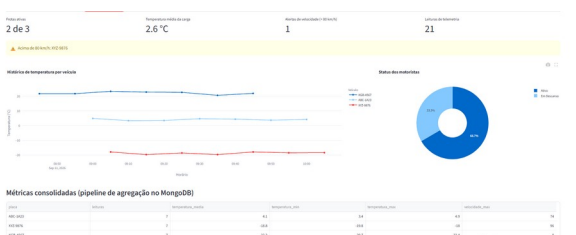
Busca por raio: 2 de 3 veículos a até 6 km da Base Centro (verde: dentro do raio; vermelho: acima de 80 km/h; cinza: fora do raio)



Depois de simular a movimentação: os caminhões andaram, deixaram rastro de rota e nenhum ficou a até 3 km de Tambaú



Antes de simular: 1 veículo a até 3 km de Tambaú



5. Como executar

```
python -m pip install streamlit pymongo pandas plotly folium streamlit-folium
python -m streamlit run geolog.py
```

Requisito: MongoDB rodando em mongodb://localhost:27017. O arquivo logitech.db é criado automaticamente na primeira execução.